Sustainable Software Engineering: Reflections on Advances in Research and Practice

Colin C. Venters[a,b], Rafael Capilla[c,g], Elisa Yumi Nakagawa[d,g], Stefanie Betz[e,g], Birgit Penzenstadler[f,g], Tom Crick[h] and Ian Brooks[i]

a Centre for Sustainable Computing, University of Huddersfield, Huddersfield, UK
b European Organization for Nuclear Research (CERN), Geneva, Switzerland
c Rey Juan Carlos University, Madrid, Spain
d University of São Paulo, São Carlos, Brazil
e Hochschule Furtwangen University, Furtwangen, Germany
f Chalmers University of Technology, Göteborg, Sweden
g Lappeenranta University of Technology, Lappeenranta, Finland
h Swansea University, Swansea, UK
i University of the West of England, Bristol, UK

Abstract

Context: Modern societies are highly dependent on complex, large-scale, software-intensive systems that increasingly operate within an environment of continuous availability, which are challenging to maintain, and evolve in response to changes in stakeholder requirements of the system. Software architectures are the foundation of any software system and provide a mechanism for reasoning about core software quality requirements. Their sustainability – the capacity to endure in changing environments – is a critical concern for software architecture research and practice.
Objective: The objective of the paper is to re-examine our previous assumptions and arguments in light of advances in the field. This reflection paper provides an opportunity to obtain new insights into the trends in software sustainability in both academia and industry, from a software architecture perspective specifically and software engineering more broadly. Given advances in research in the field, the increasing introduction of

academic courses on different sustainability topics, and the engagement of companies to cope with sustainability goals, we reflect on advances and maturity about the role sustainability in general plays in today's society. More specifically, we revisit the trends, open issues and research challenges identified five years ago in our previous paper on software sustainability research and practice from a software architecture viewpoint, which aimed to provide a foundation and roadmap of emerging research themes in the area of sustainable software architectures in order to consider how this paper influenced and motivated research in the intervening years.
Method: The forward snowballing method was used to establish the methodological basis for our reflection on the state of the art. A total of 234 studies were identified between April 2018 and June 2023 and 102 studies were found to be relevant according to the selection criteria. A further subset was mapped to the primary themes of the original paper including definitions and concepts, reference architectures, measures and metrics, and education.
Vision: The vision of this reflection paper is to provide a new foundation and road map of emerging research themes in the area of sustainable software engineering highlighting recent trends, and open issues and research challenges.

Keywords: Architectural Debt, Architectural Smells, Code Smells, Education and Training, Reference Architectures, Software Architecture, Software Engineering, Software Metrics, Software Sustainability, Sustainability, Sustainable Software, Technical Debt

1 Introduction

Software systems are omnipresent in all aspects of our lives: from health, commerce, communication, and education, to energy, entertainment, finance, governance, and defence; indeed, it has been previously claimed that "software is eating the world" (Andreessen, 2011). It is argued that software systems are one of the purest forms of creative activity undertaken by humans as their creation are not bound by the limitations of the laws of physics (Ousterhout, 2021). Increasingly, it is asserted that the value and commercial success of companies and products is determined by software and its quality (Gharbi et al., 2019).

The software crisis of the late 1960s led to more systematic approaches to the development of software systems (Royce, 1987; Boehm, 1988; Forsberg and Mooz, 1991; Schwaber, 1997; Jacobson et al., 1999) and the establishment of the field of software engineering. The first conference on software engineering was held in Garmisch, Germany in 1968 to develop guidelines and best practices for the development of software systems (NATO, 1968). While the debate on whether software engineering is a true engineering disciple, in comparison to classical engineering disciplines continues (Shaw, 2016), the field of software engineering provides "a systematic, disciplined, quantifiable approach to the development, operation, and maintenance of software" (ISO/IEC/IEEE, 2017); that is, the application of engineering to software. An alternative view of software engineering was proposed by Farley (Farley, 2021), who suggested that it is "the application of an empirical, scientific approach to finding efficient, economic solutions to practical problems". At the heart of both perspectives is the idea of measurement and quantification.

Parnas (1972) argued that the most fundamental problem in software engineering was the problem of "decomposition", i.e. how to take a complex problem and divide it into pieces that can be solved independently. Despite a range of software engineering methods that have emerged over the last fifty years, it is argued that the field of software engineering has been in a methods war for over forty years and has still not fully understood how to successfully construct successful software systems on a repeatable basis that does not result in software projects being significantly over time and budget or failing as a result of their size and complexity (Jacobson, 2018). Based on the estimation accuracy of cost, time, and functionality, it is estimated that approximately 31.1% of projects will be cancelled before they are completed and that 52.7% of projects will cost 189% of their original estimates as a result of poor decision latency (Johnson, 2021). While the findings and methodology of the Standish

Group have been challenged (Eveleens and Verhoef, 2010), it continues to paint a rather bleak picture of the high rates of software project failure, which has resulted in the delivery of low-quality products, schedule delays, and unplanned expenses (Standish Group International, 2020).

$$C = \sum_{p} cp\ tp \tag{1}$$

Complexity, from a software engineering perspective, is "anything related to the structure of a software system that makes it difficult to understand and modify" (Ousterhout, 2021). Equation (1) suggests that the overall complexity (C) of a system can be determined by the complexity of each component $p(c_p)$ weighted by the fraction of time developers spend working on it (t_p). Common symptoms of complexity include change amplification, cognitive load, and "unknown unknowns". As such, complexity presents threats and risks to the sustainability of software-intensive systems as it leads to a range of rotten symptoms, including software rigidity, fragility, immobility, and viscosity, that result in non-trivial maintenance and high evolution costs (Thomas, 2016; Lilienthal, 2019). Software design is a key component in addressing complexity, which starts with software architecture, as it represents the set of structures required to reason about the system and comprises both software elements, their properties and relationships (Bass et al., 2021). All software-intensive systems have an architecture even when it is not explicitly designed. However, these accidental software architectures (Booch, 2006) are underpinned by sub-optimal design decisions that exacerbate complexity, which are the foundation for software decay and death in all software investment (Martin, 2017). As a result, sustainability has emerged as a growing area of interest in the field of software engineering, requirements engineering, and software architecture with a growing body of research, which focuses both on the software system itself and how software systems can be utilised to address sustainability (Alharthi et al., 2018; Condori-Fernández et al., 2018; Mourão et al., 2018; Oyedeji et al., 2018; Seyff et al., 2018; Duboc et al., 2019; Lago et al., 2020; Pham et al., 2020; Andrikopoulos and Lago, 2021; Gupta et al., 2021; Pham et al., 2021; Bischoff et al., 2022; Fatima and Lago, 2023; Funke et al., 2023; McGuire et al., 2023). Its importance as a topic has been further underlined by ongoing initiatives from the US National Science Foundation (NSF, 2022) and the UK Engineering and Physical Sciences Research Council (EPSRC, 2020, 2021) to help develop research software as a sustainable, first-class, experimental scientific instrument (Goble, 2014). This has resulted in the establishment of a Software Sustainability Institute (SSI, 2010) and the Society of Research Software Engineering (SRSE, 2013) in the UK, which has seen increasing internationalization of the field of Research Software Engineering to countries outside of the traditional base in Europe such as RSE@SUN in South Africa (Martin, 2022).

Software sustainability is generally understood as "the capacity of a socio-technical system to endure" (Becker et al., 2015). In 2018, we presented our perception of the maturity of software sustainability from a software architecture point of view (Venters et al., 2018). This article argued that software architectures are fundamental to the development of [technically] sustainable software systems as they are the primary carrier of architecturally significant requirements (ASRs), such as extensibility and maintainability (Cervantes and

Kazman, 2016). As such, software architectures manifest the major design decisions that determine a system's initial development, deployment, and evolutionary change; none of which can be achieved without a unifying architectural vision. Software architectures strongly influence sustainability, because they affect how developers can understand, analyse, extend, test, and maintain a software system.

While a number of communities have attempted to address the challenges of achieving sustainability from their different perspectives, there is a severe lack of common understanding of the fundamental concepts of sustainability and how it relates to software systems (Venters et al., 2021). Consequently, in this reflection paper, we revisit the main contributions of the original paper against the current and evolving status of the state of the art and practice from a broader software engineering viewpoint. We reflect on different approaches and techniques to achieve sustainability, as well as metrics and key performance indicators (KPIs) to measure sustainability in different dimensions so they can be used in academia and industry settings. This lays the foundation to elaborate a road map for the future. The remainder of this paper is structured as follows. In the first instance, we outline the methodological aspects of this paper and the process we took as part of conducting it. In the sections that follow, we review the definition of software sustainability and underlying assumptions, discuss approaches related to software architecture sustainability with a focus on the role of reference architectures, outline software metrics related to estimating sustainability in architecture and code, and revisit education and skills regarding recent educational experiences in teaching sustainability concepts. The paper concludes by identifying a number of open issues and research challenges.

2 Methodology

The purpose of this reflection paper is not to conduct a deep analysis of the state of the art on sustainable software engineering since 2018. The primary aim of this paper is to reflect on how the state of the art was influenced by the contributions set out by Venters et al. (2018). To achieve this, we adopted a forward snowballing approach to systematically examine all the citing work of the original paper as part of this reflection process. Forward snowballing is the process of identifying new studies based on those studies citing the original study being examined. We followed the guidelines for snowballing outlined by Wohlin (2014). The value of forward snowballing in maintaining the currency of systematic reviews has been demonstrated to have higher precision and a slightly lower recall; however, the risk of missing relevant studies should not be underrated (Felizardo et al., 2016; Wohlin et al., 2022). We identified 84 Scopus and 150 Google Scholar citations between April 2018 and June 2023; 234 citations in total. We defined the following inclusion criteria for each study:

- IC_1: Study cites Venters et al. (2018).
- IC_2: Study focuses on software architecture or software engineering.
- IC_3: Study is related to one or more sustainability dimensions.
- IC_4: Study is peer-reviewed (journal articles, conference and workshop papers, magazine articles).

A study was excluded if one or more of the following criteria were satisfied:

- EC_1: Duplicates of the same study.
- EC_2: Study not written in English.
- EC_3: Full-text study is not publicly accessible.
- EC_4: Short study less than four pages long.
- EC_5: Non-scientific and non-peer-reviewed artefacts (e.g., posters, thesis, or books).

This resulted in an initial set of 102 studies after filtering. In addition, we classified the selected studies using keywords and abstracts, to further filter the results in order to map studies to the original themes of Venters et al. (2018) and identify new topics. The main themes are shown in Table 1. This resulted in a subset of studies after filtering.

Table 1: Studies discussing the notion of sustainability and other sustainability-related topics

Topic	No. of Publications
Sustainability Definitions	102
Reference Architectures	2
Metrics and Measurement	12
Education	3

All studies were included in the sustainability definitions category since the all focused on sustainability. The first author then conducted an initial classification of the studies. In order to minimise bias in the classification, the primary lead of each section of the study then reviewed the studies independently. Disagreements regarding the classification between co-authors concerning the inclusion and exclusion of a study were resolved in iterative consensus meetings mediated by a third author who also read through all studies.

2.1 Threats to Validity

As with all studies, this paper has certain limitations. We discuss the validity and limitations of this type of study proposed by Wohlin (2014). Internal validity is the extent to which you can be confident that a cause-and-effect relationship established in a study cannot be explained by other factors (Creswell and Guetterman, 2020). To address this, we followed the guidelines for conducting snowballing suggested by Wohlin (2014). External validity is the extent to which the findings of a study can be generalised (Creswell and Guetterman, 2020). The main limitation of our analysis is that it is based on the application of a single technique, i.e., forward snowballing. As a result, further analysis will be needed using a range of combined techniques to provide a more in-depth

treatment of the topic. However, this approach has provided some valuable insights and indication as to how the field has changed. Construct validity is crucial to establishing the overall validity of a method (Creswell and Guetterman, 2020). In this study, the inclusion and exclusion criteria can be considered a potential confounding factors. However, as suggested by Kitchenham et al. (2009), it was documented and its application was extensively discussed among the authors.

3 Software Sustainability: Definitions

Software sustainability before 2018: What does software sustainability mean in the field of software engineering? We observed in 2018, that there was no agreed definition of the concept of software sustainability. However, this is not a problem unique to the field of software engineering (Sverdrup and Svensson, 2005; Glavic and Lukman, 2007). Garlan et al. (Garlan et al., 2010) noted that there is also no universal definition of software architecture. Since its inception, the Software Engineering Institute has collected more than one hundred and fifty definitions of the term software architecture from the literature and practitioners around the world (SEI, 1984). However, we reiterated the argument put forward by Seacord et al. (Seacord et al., 2003) who stated that before software sustainability can be measured, it must be understood. To this end, we examined the range of definitions available starting from a basic understanding of how the term sustainability is defined. The Oxford English Dictionary defines sustainability as "the quality of being sustained", where sustained can be defined as "capable of being endured" and "capable of being maintained" (OED, 2023). Endured is defined as "continuing to exist" and maintained as "being supported" (OED, 2023). This suggested that the concept of longevity as an expression of time and the ability to maintain were key factors at the heart of understanding sustainability. However, from a software engineering perspective, defining software sustainability as its capacity to endure is simplistic and requires greater precision if we are to engineer software systems. It is worth noting that other engineering disciplines would refer to this quality as durability (Bijen, 2003). Reframing the discussions around this might serve for clarity if the field of software engineering also adopted the same terminology (Atkins and Escudier, 2013).

In contrast to viewing software sustainability as a measure of endurance, a number of definitions aligned software sustainability to one or more software quality attributes that we suggested contributed to the sustainability of the software artefact including maintainability and extensibility (Seacord et al., 2003; Calero et al., 2013; Penzenstadler, 2013; Groher and Weinreich, 2017). Our original view on the definition of software sustainability was influenced by Venters et al. (Venters et al., 2014) who defined software sustainability as a "composite, non-functional requirement which was a measure of a system's extensibility, interoperability, maintainability, portability, reusability, scalability, and usability". However, one of the principal challenges in defining sustainability as a non-functional requirement is the need to demonstrate that the quality factors have been addressed in a quantifiable way. With the exception of Seacord (Seacord et al., 2003), none of the proposed definitions suggested appropriate metrics that could be employed to measure the sustainability of a software system. Never-

theless, we continue to argue that maintainability and evolution of the software artefact are key enablers to achieving long-living software (Durdik et al., 2012).

Software sustainability after 2018:

Our analysis strongly suggest that there is no consensus on the term "software sustainability" or "sustainable software". Studies adopt a wide range of definitions from the "capacity to endure", to one or more "software qualities" (Hahner et al., 2019), to focusing on one or more of the "dimensions of sustainability" (Tanveer, 2021; Andrikopoulos et al., 2022; Garc´ıa-Bernâ et al., 2022). McGuire et al. (McGuire et al., 2023) highlight that the most commonly considered dimension is ecological sustainability, followed by technical, economic, social, and individual. In addition, they argue that sustainability is a multisystemic, stratified concept that applies to both the software development process and the resultant software artefacts. While there continues to be a small number of contributions to formalise a definition of software sustainability, others continue to muddy the waters. Cohen et al. (2021) exemplify the issue, where they argue that maintainability, [sustainability], and robustness are core aspects of building [sustainable software], which is underpinned by ensuring a [sustainable] approach to software development.While few would argue against maintainability being a core internal quality of a software system, the terms sustainability and sustainable are not defined in context, leaving them open to [mis]interpretation. In contrast, Venters (2021) now argues that sustainable software is that which is "explicitly designed for continuous maintainability and evolvability without incurring prohibitive technical debt and a negative impact on the dimensions of sustainability". What remains is the need to align this definition with a core set of software metrics, such as average component dependency, propagation cost, structural debt, duplicated lines of code, cyclomatic complexity, cognitive complexity etc., combined with change history metrics (von Zitzewitz, 2022).

As such the concept remains an elusive and ambiguous term with individuals, groups, and organisations holding diametrically opposed views (Venters et al., 2021). This lack of clarity ultimately leads to confusion, and potentially to ineffective and inefficient. Since our earlier work (Venters et al., 2018), we have observed significant progress on estimating sustainability at an architectural-level and other related technical areas including technical debt and code smells (Sharma and Spinellis, 2018; Azadi et al., 2019). Other sustainability dimensions offer more clear guidelines to evaluate sustainability. For instance, environmental sustainability provides measures to estimate energy savings, while social sustainability uses indicators for efficient transportation (Sdoukopoulos et al., 2019).

Regarding technical sustainability, and more specifically the software architecture area, we found some basic estimators about the longevity and stability of software designs. While there are plenty of metrics to estimate maintainability indicators in source code, there is still a lack of metrics and good KPIs that can explain how sustainable a software system or an architecture is. The right combination of existing metrics to derive sustainability reference values is still challenging and an open research area. This problem is exacerbated by the lack of cost models to estimate capturing the key design decisions that lead to a software architecture. As a consequence, software sustainability research is only scratching the surface in providing good indicators to estimate

the sustainability of systems and their impact on other related sustainability dimensions. As a result, implementing any of the 17 United Nations Sustainable Development Goals (SDGs) in organisations results in complexity because of the lack of metrics to estimate these in various contexts (Seyff et al., 2022). There are well-established sustainability reporting approaches which may also be a basis for metrics, for example, the Global Reporting Initiative indicators, as well as nation-specific approaches such as the Well-being of Future Generations Act (Future Generations Commissioner for Wales, 2015) in Wales, UK.

4 Software Architecture Sustainability

Architecture sustainability before 2018: When we published our previous work in 2018 (Venters et al., 2018), architecture sustainability was drawing attention as a research topic aligned with the emerging interest in software sustainability. Architecture sustainability refers to the degree to which the architecture supports its continued maintenance and evolution over time without requiring substantial and expensive restructuring (Koziolek, 2011a). It also refers to the ability to tolerate changes resulting from changes in requirements, environment, technologies, business strategies, and goals throughout the system life cycle (Avgeriou et al., 2013). Architecture sustainability was also considered a broader quality attribute that aggregates other qualities, such as maintainability (i.e., understandability, analyzability, stability, testability), modifiability, portability, and evolvability (Venters et al., 2018).

It is recognised that software architectures as a foundation of any software systems supporting analysis of quality attributes, including maintainability and extendability (Garlan, 2000), can promote the achievement and existence of sustainable software systems. Hence, sustainability should be explicitly considered in the system design, even when sustainability is not a primary purpose of that system (Becker et al., 2014). Also, adequate architectural decisions and their effective management directly impact the sustainability of both software systems and their architecture.

At the same time, it lacked a consensus on what architecture sustainability was precisely and even how to measure it, while the notion of architecture sustainability focused on technical perspectives of assuring maintainability and extendability (Amri and Ben Saoud, 2014) and, as a consequence, long-term use of systems in a continuous changing execution environment. Two related phenomena could degrade such sustainability (architectural drift and erosion (Taylor et al., 2009)) caused by many factors, with different solutions being investigated, including architectural technical debt management ("sustainability debt" (Betz et al., 2015; Ojameruaye et al., 2016)), measurement of architectural design sustainability (Koziolek, 2011b; Koziolek et al., 2013), and sustainable architectural design decisions (architectural knowledge sustainability) (Capilla et al., 2017). Other approaches to promote architecture sustainability also appeared in that time (Koziolek et al., 2012). Software reference architectures were also considered as a means to support sustainable architecture designs, as they encompass reusable architectural knowledge of key design decisions and provide a common vocabulary and blueprint for concrete architectures in a specific domain (Nakagawa et al., 2011).

Finally, we had foreseen that achieving sustainable architectures would require capturing significant sustainable design decisions and their rationale. A key challenge was how to capture a minimalistic set of design decisions combined with lightweight software development approaches. We also highlighted sustainability would be a challenge for the field of software architecture in the future years.

Architecture sustainability after 2018: Since 2018, numerous approaches have been investigated by the research community addressing architecture sustainability (Andrikopoulos et al., 2022). The practices for such sustainability have included consideration of the other sustainability dimensions (environmental, social, economic, and individual), aligned to the investigations of these dimensions from the software engineering perspective, such as environmental sustainability in requirements processes (Garc´ıa-Mireles and Villa-Mart´ınez, 2018), energy consumption for sustainable software engineering (Moises et al., 2018), individual sustainability in sustainable software engineering (Nazir et al., 2020), interaction between environmental sustainability and software quality (Garc´ıa-Mireles et al., 2018), and social sustainability as a concern in software architecture design (Volpato et al., 2019).

These other dimensions can impact technical sustainability, while systems complying with a specific software architecture can impact these other dimensions (Betz et al., 2022), as illustrated in Figure 1. For instance, AUTOSAR, a reference architecture for the automotive sector (Autosar, 2023), provides us with examples of both effects. The transition away from fossil fuel use, for environmental reasons, has affected the technical sustainability of AUTOSAR. A study indicates AUTOSAR remains sustainable on the technical dimension (Wang and Yin, 2019), and software vendors are offering AUTOSAR-compliant electric vehicle software, e.g., KPIT (KPIT Technologies Ltd, 2022). Clearly, AUTOSAR-compliant fossil fuel vehicles could impact the other dimensions of sustainability through greenhouse gas (GHG) emissions and particulates. One of the objectives of the AUTOSAR project is that "AUTOSAR shall support sustainable utilization of natural resources" (Autosar, 2023), but there is still an absence of published research on how effectively this architecture has satisfied this objective.

[width=1]Architecture sustainability v1.0.jpg

Figure 1: Interactions of Technical dimension of sustainability with other dimensions

There has been an urgent need to develop agreed reference architectures for systems that significantly impact broader sustainability dimensions. The literature has shown progress in areas such as agriculture (Giray and Catal, 2021; Kakamoukas et al., 2021), e-health (Grua et al., 2020), and smart cities (Santana et al., 2018).

In another perspective, the possibility of continuous updating of reference architectures has been recognized as one of the critical factors for their sustainability, i.e., the ability of these architectures to tolerate changes needed to remain aligned with a target domain or goal, and updated and useful for the long-term in a given domain. Some remarkable examples of technically sustainable reference architectures are AUTOSAR, AXMEDIS (AXMEDIS, 2023), and

ARC-IT (US Department of Transportation, 2022), and some reference architectures for Industry 4.0 (Nakagawa et al., 2021). AUTOSAR started almost 20 years ago and currently presents very extensive documentation organized into four phases and several releases associated with four versions. ARCT-IT is already 27 years old and, during this time, there have been nine versions with several modules changed, reallocated, added, or removed to cover more than 6,000 requirements and around 130 standards. AXMEDIS, an architecture for automating the production of cross-media content, has a life cycle of over 15 years with three big releases with updates in standards to the content management of partners, such as BBC and HP, and refinement of their descriptions with new viewpoints. These architectures are supported by consortia of strategic partners; for instance, AUTOSAR is currently maintained by a core of automotive manufacturers, such as BMW Group, Stellantis, Toyota, Ford, and Volkswagen.

Major IT services providers, e.g., Google, Amazon, and Microsoft, have also paid attention to the other dimensions of sustainability by including them in their architecture tools. Google's Cloud Architecture Framework includes a section on designing for environmental sustainability (Google, 2022). One pillar of the AWS Well-Architected Framework is titled sustainability (AWS, 2022). The Microsoft Azure Well-Architected Framework includes a discussion of sustainable workloads (Microsoft, 2022). However, all of them are still narrow in their conception of sustainability with the discussion being constrained to the GHG emissions associated with cloud workloads.

The practice and the state-of-the-art research have recognized the value of software architecture as an essential enabler of sustainable software systems, i.e., it is necessary to architect for sustainability (Lago et al., 2022) The achievement of architecture sustainability has been then a concern for many years but still continues to be a challenge for the software architecture field.

5 Metrics and Sustainability Goals

Metrics before 2018: In 2018 we argued the existence of a plethora of software metrics that could be used to estimate technical sustainability. At that time most of the software metrics were devoted to evaluating different aspects such as maintainability and code complexity, modularity (Voas and Kuhn, 2017) and architecture sustainability (Koziolek et al., 2013; Le et al., 2016) among others. Nevertheless, the difficulty to estimate sustainability in software systems many times comes from the right combination of the existing metrics and defining appropriate indicators as good sustainability indicators.

Also, there are several quality attributes (e.g., maintainability, stability, modularity, extensibility, scalability) that are relevant to evaluate the sustainability of a system from different angles, but we need to estimate these attributes using the right combination of metrics. Additionally, the longevity of systems as an indicator of good evolution (Durdik et al., 2012) and the capacity of endurance as a clear indicator of the resistance of a software system versus changes, both are two important qualities that clearly affect to sustainability (Penzenstadler et al., 2014; Becker et al., 2015).

Other kinds of metrics understand software sustainability as an evaluation of the ripple effects in architecture design decisions (Capilla et al., 2017) and

analyse the effects in software systems when decisions change. Thus, maintaining a sustainable set of decisions captured and the relationships between them is another challenge when building software-intensive systems.

Another group of metrics are used to evaluate the greener aspects of systems such as energy savings and addressing the challenges of the environmental dimension. These metrics seem to be easier to define versus those addressing technical sustainability concerns because we can analyse the energy consumption of systems (or other kinds of sustainability savings such as waste management systems) using the data coming from sensors or hardware usage (e.g., the Python programming language is much more pervasive using hardware resources than Java). Broadly speaking, these sustainability indicators are very dependent on the target domain or system. Green approaches for sustainability like the GREENSOFT reference model (Naumann et al., 2011) suggested the term green and sustainable software for resource and power consumption of ICT. One more modern work provides a comparative analysis of GREEN ICT maturity models (Lautenschutz et al., 2018). Also, the GREENS workshop series have investigated software engineering challenges and metrics to estimate the green and sustainable aspects of software systems (Lago et al., 2013). A taxonomy of metrics for technical and environmental sustainability dimensions can be found at (Oyedeji et al., 2018).

Metrics after 2018: Despite the many metrics used for years to estimate different aspects aimed to improve the maintainability of systems and architectures, there is still a need to count on adequate sustainability indicators that can prove a system is sustainable over time. One work that analyzes the relationship between sustainability and possible metrics to evaluate the ripple effect of changes in architectural design decisions is discussed in Carrillo and Capilla (2018). The authors suggest three instability ranges to find out those stable decisions that can affect to the sustainability of the underlying architecture. A related effort aimed at understanding the complexity of the architectural design decision networks is discussed in Carrillo et al. (2020), where the authors analyse the complexity of decision graph topologies according to the cycles and the different paths to reach a particular decision. The authors evaluate the proposed model using two service-based case studies and four open-source projects and they highlight the relation to technical sustainability based on the evolution of the architectural changes based on the complexity of the decision networks. Nevertheless, it is still hard to prove the connection between these metrics and their influence on the sustainability of a system.

Regarding the estimation of software sustainability, it is hard to define which software quality metrics and a combination of them matter. As argued in Condori-Fernández et al. (2018), *"measuring the sustainability of software products is sill in the early stages of development"*, so the authors developed a platform and a dashboard called MEASURE aimed to find out how the quality attributes related to the technical sustainability can be measured. They suggest a number of metrics to operationalize which quality attributes can contribute to technical sustainability and they report the important qualities via interviews with focus groups in a workshop. Additionally, the importance of these qualities is discussed in Condori-Fernández and Lago (2018) as the authors run a sustainability survey to determine the quality characteristics and sub-characteristics that are relevant for pairs of sustainability dimensions. Among these, the au-

thors report for technical sustainability those qualities that contribute with different degrees (e.g., greater than 60%, between 40% and 60%, etc.) so they can rank the qualities per each dimension and define relationships between qualities and dimensions (e.g., shared qualities for more than one dimension). The work reports that the pairs "technical-economic" and "technical-environment" exhibit higher dependencies.

In addition, software maintainability metrics (e.g., complexity, coupling, etc) are a good choice to estimate software sustainability indicating how good or poor is the quality of our system Gradinik et al. (2020) Karanikiotis et al. (2021), as high maintenance costs can be perceived as low sustainability. Another way to estimate the sustainability of a software system indirectly is to analyse the appearance of architectural and design smells that negatively impact its architecture. Some recent progress is the implementation of architectural and design smell metrics in the Designite tool (Sharma et al., 2020) which can be used to evaluate a loss of quality along the evolution of software designs as indicator of low sustainability and architectural decay. Despite the little progress made to provide sustainability formulas offering clear indicators of good or bad sustainability of a software system, most of the current efforts go in the direction to analyse the quality of systems via technical debt analysis tools, and use these results to improve quality defects as a way to enhance software sustainability (i.e., seen as a reduction of the maintainability effort). The work from (Guamán and P´erez, 2021) suggests a meta-model (i.e., ARCMEL) and tool (i.e., ARCMEL SCAT) supporting metrics to identify green decisions related to code, design, and energy smells in order to estimate software and environmental sustainability.

From the perspective of the sustainability environmental dimension and the green metrics used up to 2018 in a broad number of domains (e.g., software, energy, waste and water management, transportation), there are new initiatives and indicators that help to estimate the environmental footprint and reduce its impact in organizations. As we cannot cover the green aspects in all domains, we will mention some of them. Greening software via green IT and green software engineering initiatives is a way to define and understand several sustainable indicators as best practices, such as sustainable programming languages that can reduce CPU consumption (e.g., Java is more sustainable than Python), principles for building green or sustainable software[1], or move the software to the Cloud.

Recent studies (Mancebo et al., 2021a) analyse the relationship between maintainability and energy consumption. The authors investigate potential energy savings in several open-source releases that went under a refactoring or maintenance process. They measure the energy consumption of software using the proposed Framework for Energy Efficiency Testing to Improve environmental Goals of the Software (FEETINGS) (Calero et al., 2021; Mancebo et al., 2021b). Therefore, we can establish a direct clear connection between technical and environmental sustainability dimensions on behalf of identifying those metrics that measure different sustainability concerns (e.g., software maintainability and energy consumption metrics) but are related. Other works (Hintemann and Hinterholzer, 2019) analyse the green aspects and energy consumption of cloud data centers, which is a big problem nowadays as more and more data centers

[1]https://greensoftware.foundation/

are becoming energy predators. These results are also very dependent on the programming language chosen (Koedijk and Oprescu, 2022).

Other organizations suggest a set of metrics to estimate sustainability and several green indicators in worldwide cities and universities[2], and use these indicators to rank the most sustainable universities according to different indicators such as energy consumption, water management, efficient transportation and so on. These efforts are typically supported by green offices established in universities. For instance, the green office of Rey Juan Carlos University (Spain) promotes efficient and sustainable transportation of students to the university campuses using the carpooling software[3] share vehicles and hence, reduce the carbon footprint. Therefore, adequate KPIs for measuring sustainability in the environmental dimension are still an open challenge.

6 Sustainability Education and Skills

Education and skills before 2018: In 2018, we addressed education and skills in two aspects: (i) Education and training (for example, professional learning and development) are part of individual sustainability — the ability of a person to maintain and evolve themselves over their lifetime; and (ii) While there had been work on integrating sustainability into undergraduate computing education (Cai, 2010; Fisher et al., 2016), education on sustainable software was lacking, both in formal academic education, as well as capacity-building for research and innovation. We had seen the start of major national and international educational reform initiatives focused on digital, data and computational skills from a wider societal perspective (Tryfonas and Crick, 2015), and more specifically, computer science education (Brown et al., 2014; Moller and Crick, 2018), as well as wider scrutiny of relevant pedagogy and practice (Davenport et al., 2016; Murphy et al., 2017; Simon et al., 2018). Interventions to address the lack of skills in research software include scaling of initiatives such as Software Carpentry (Wilson, 2014), the Software Sustainability Institute (Crouch et al., 2013), the WSSSPE (*Workshop on Sustainable Software for Science: Practice and Experiences*) workshop series[4], major nationally funded initiatives such as the Institute of Coding in the UK (Davenport et al., 2020), and wider work on raising the status of software in research (Smith et al., 2016; Crick et al., 2017; Sufi and Jay, 2018).

Education and skills since 2018: In 2020, the COVID-19 global pandemic led to a significant shift in how education was carried out, across all settings and contexts, with a rapid shift to remote online education (Crick et al., 2020b; Watermeyer et al., 2021a). Apart from the social, cultural and economic toll, mental health challenges were on the rise (Crick et al., 2022a), and everyone was stretched into adapting to new ways of learning, teaching and assessment (Prickett et al., 2020; Crick et al., 2022c).

We now have education back on campus, largely face-to-face, but we have seen a significant difference in the student's behaviour in class who may be physically going to university for the first time in their programmes. There

[2]https://greenmetric.ui.ac.id/
[3]https://hoopcarpool.com/urjc/
[4]https://wssspe.researchcomputing.org.uk/

is less interaction and less networking, and a worry that this may affect their opportunities in terms of starting their career starts. Even in the context of using COVID-19 as a catalyst for positive change (Crick, 2021), the continuing impact on education is clear (Watermeyer et al., 2021b; Hardman et al., 2022; McGaughey et al., 2022); this is keenly felt in computer science and software engineering education, from the impact on emerging national curricula and qualifications (Crick, 2022; Sentance et al., 2022), impact on diverse student groups (Crick et al., 2022d,b, 2023), future models of learning, teaching and assessment (Siegel et al., 2021; Watermeyer et al., 2022; Cutts et al., 2023), micro-credentials and new taxonomies for personalised learning and skills development (Ward et al., 2021, 2023), concerns with the robustness of digital infrastructure and cyber resilience of institutions (Irons and Crick, 2022; Prickett et al., 2023), through to degree accreditation and professional recognition (Crick et al., 2020a; Irons et al., 2021). Furthermore, we are also seeing the wider emerging impact of artificial intelligence on education, across all settings and contexts, with broad dimensions of software and sustainability to consider (Dwivedi et al., 2021, 2023).

Specifically for sustainability education, there is an increased focus at various institutions globally to incorporate material that relates to the UN Sustainable Development Goals[5]. Engineering is asked to deliver in terms of contributions to the SDGs, and a UNESCO report highlights the crucial role of engineering in achieving each of the 17 SDGs. It shows how equal opportunities for all are key to ensuring an inclusive and gender-balanced profession that can better respond to the shortage of engineers for implementing the SDGs[6]. This is also supported by the paper from Paulauskaite-Taraseviciene et al. (2022). In this recent work, the authors investigate the education for sustainable development in engineering programs. They found very few study programs related to the development of sustainability in computer science, mathematics or physics. The authors argued this might be due to the fact that it is difficult to integrate and that the focus in those subjects is more on technical skills and rigorous classical theory e.g. Paulauskaite-Taraseviciene et al. (2022).

Nevertheless, for technology development, there are clear themes in government policy that require anchoring sustainability skills in the digital/tech workforce; for example in the UK, leading with: *"As well as short-term planning for your projects requirements, you should aim to create a long term plan on how to increase the sustainability of your technology or service across its entire lifecycle."*[7].

In addition, we contributed to a systematic literature review (Peters et al., 2023) with other colleagues with an interest in how sustainability to understand how it is being taught in an educational environment in order to uncover the key sustainability topics, skills, and competencies. The primary studies report on specific modules through which sustainability education is implemented, e.g., teaching general sustainability competencies or cross-cutting skills like systems thinking. The studies provide guidance for module design, for instance, thesis projects about sustainable engineering. There were efforts to teach sustainability in IT via one or several workshops or seminars, assignments, projects, or modules, for example, green computing, introductory modules on software and

[5]https://www.unesco.org/en/education/sustainable-development
[6]https://unesdoc.unesco.org/ark:/48223/pf0000375644.locale=en
[7]https://www.gov.uk/guidance/make-your-technology-sustainable

sustainable development, final year project module on sustainability, "software engineering sustainability", and ethics. This was further complemented by a parallel study which investigated aims to identify the industrial sustainability needs for education and training from software engineers' perspective (Heldal et al., 2023). The results suggested that educational programs should include knowledge and skills in core sustainability concepts, system thinking, soft skills, technical sustainability, building the business case for sustainability, sustainability impact and measurements, values and ethics, standards and legal aspects, and advocacy and lobbying.

Most of the existing courses we found in the literature do not address the severity of challenges and demand for unprecedented change. However, many universities do make an effort of including sustainability somewhere as an aspect of a course. With the exception of emerging work in this space (Frezza et al., 2019; Swacha et al., 2021), education is mostly conceived of as training predefined knowledge, skills or competencies. Activism in and through education in computing education is hardly discussed, but this is starting to change, e.g., Lachney et al. (2021).

Looking ahead, one key question is how to create space for sustainability education in computing curricula; for example, embedding it across the knowledge and competency models for the new ACM/IEEE-CS/AAAI Computer Science Curricula (CS2023)[8] (and related ACM curricula recommendations[9]), as well as explicitly in national-level work such as the QAA Subject Benchmark Statement for Computing in the UK[10]. However, with an understanding of education as personal evolution, related research could capture individual growth or more sustainable ways of living, or relating to the world (Peters et al., 2023).

7 Outlook and Future Directions

In this section, we outline some future trends regarding the discussion of the previous sections as a way to understand how sustainability has evolved since 2018 and what we expect for the upcoming years. We summarize the evolution of sustainability since 2015 and our expectations for the upcoming years in Figure 2.

Trends in software architecture sustainability: While the process of designing software architectures is relatively well established (Cervantes and Kazman, 2016; Bass et al., 2021) - if not implemented in practice - methods for reasoning and measuring architectural consistency and architectural structures, which erode sustainability are open research challenges. Architectural consistency aims to measure the architectural drift or erosion of the implemented architecture against the designed architecture. Several techniques have been proposed, which can be classified based on how they extract the implemented architecture using static or dynamic code analysis. However, these approaches have several limitations, including being driven by the codebase, not the architecture, assessing the impact of architectural flaws on the recovered architecture, and the difficulty in quantifying architectural inconsistency effects (Ali et al., 2018).

[8]https://csed.acm.org/vision-statement/

[9]https://www.acm.org/education/curricula-recommendations

[10]https://www.qaa.ac.uk/the-quality-code/subject-benchmark-statements/computing

In practice, software architectures are often documented after implementation (Jansen et al., 2008). Architectural reconstruction and recovery aims to reverse engineer the software architecture from system artefacts in order to facilitate analysis and understanding. Approaches are either based on behaviour-based clustering techniques such as weight or program slicing or based on filtering methods, which recover partial descriptions of the architecture. However, current techniques are limited to extracting code-level information, not architecturally significant abstractions. In addition, the approaches are context-dependent, and no method focuses on those aspects of the architecture which address software qualities (Tamburri and Kazman, 2018). New approaches are required to recover relevant views of the software architecture, including architectural design decisions, that map to architectural concepts, i.e., patterns and tactics (Mirakhorli, 2015).

Whilst we expect work on the technical sustainability of software architecture to continue to develop on the firm foundations discussed in Section 3, researchers will need to focus their attention on the interactions with the other dimensions of sustainability. Firstly to ensure that software architectures are resilient in a period of rapid change as the global economy transitions away from fossil fuels. Secondly, software architectures are designed to support sustainability gains in the application areas for which they are designed. This may require engagement with a broader range of stakeholders than previously expected. Except for some established reference architectures like AUTOSAR, ARC-IT, and AXMEDIS, and considering most other architectures have not yet survived after their first release or publication (Garc´es et al., 2021), the sustainability of these architectures themselves is an open issue to be investigated yet (Nakagawa and Antonino, 2023; Volpato et al., 2017). There is also a gap in the research literature regarding assessing the sustainability impacts of existing reference architectures, such as AUTOSAR and Continua.

Finally, it is necessary to better understand the relations and impacts of the different sustainability dimensions on the software architecture design and how to address them jointly to comply with both the sustainability and the project's goals.

Trends in sustainability metrics: Although the ISO website claims that the draft ISO/IEC/DIS 25010 standard [11] contributes to UN Sustainable Development Goal 9, it is unclear how such a connection should be implemented. Despite the recent advances in sustainability metrics in specific application domains, we identified some gaps that must be covered in future research works, such as we summarize below:

- Define high-level technical sustainability indicators based on code and architecture metrics that can provide additional insight into the endurance of a software system regarding its maintenance cost and not only based on classical maintainability metrics. While measuring sustainability in code seems straightforward, analyzing the sustainability of an architecture or system is much more complex. Therefore, having new metrics connecting better code and architecture quality indicators could lead to defining more accurate sustainability indicators and including the development and maintenance costs.

[11] https://www.iso.org/standard/78176.html

- Although environmental sustainability seemed a more mature research area because energy, carbon footprint, water and waste management have clear metrics, we encourage additional research on sustainability predictor models that can anticipate unexpected demands of resources. This could add more intelligence to the current metrics by analyzing the consumption of resources and adapting these to peaks in demand.

- We need KPIs to estimate sustainability in various domains (e.g., climate, transportation, water, oceans) as most of the measures are aimed to estimate sustainability in software, energy, and waste management. Thus, organizations that want to launch sustainable initiatives demand concrete indicators and rankings to estimate the level of sustainability based on low-level metrics.

- There is still a significant lack of metrics to measure the sustainability in individuals and social communities, mainly as a result of sustainability actions in technical and environmental dimensions. We still need to connect these dimensions to understand how sustainability benefits people.

In summary, we need a renewed set of metrics properly combined that can estimate sustainability in the different dimensions but also how these dimensions connect to each other in order to address several SDGs.

Trends in sustainability education and skills: Scanning job openings and descriptions, the skills demanded by specialists in sustainability include analytical skills, communication skills, leadership skills, and problem-solving skills, as provided by programs like a Bachelor of Science in Sustainability[12]. Sometimes, the term sustainability engineer is simply a synonym for environmental engineer, other times it has a much wider scope. The open challenge is how to integrate a sufficient amount of this expertise into a computing education, specifically when there is a given area of expertise like software architecture. It is back to the old question of how to strike a balance between generalist (knowledge of computing and sustainability) and specialist (e.g., software architecture and sustainable software infrastructure).

Another topic of relevance is ethical responsibility and greenwashing. There is a significant risk of greenwashing and enthusiastic company mission statements that are not backed by effective actions. Or it is only intercepted by a single sustainability expert responsible for a multi-billion dollar company who can only do so much, e.g., organise sustainable energy use. Here, a cross-cutting education covering all CS courses could also help, as could certification by independent institutions. Furthermore, it is highly relevant to deepen the understanding of the interplay between sustainability and digitization. This might be in part addressed by the announcement by the ACM (the world's largest educational and scientific computing society, whose aim is to advance computing as a science and profession) of a number of ACM Presidential Task Forces in February 2023 (Ioannidis, 2023). One of these directly focuses on how ACM members can offer their expertise in the multidisciplinary efforts toward achieving the 17 UN SDGs (as well as the Paris Agreement on Climate Change).

[12]https://online.maryville.edu/online-bachelors-degrees/sustainability/careers/sustainability-specialist-job-description/

[width=1]overview.png

Figure 2: Evolution of sustainability trends since 2015 and agenda towards 2030

Software design is a key component of sustainable software, which starts with software architecture (Durdik et al., 2012). Sub-optimal software design, if any, leads to accidental complexity, technical debt, code smells, and an increase in the risk of software entropy, which results in prohibitive high-maintenance and evolution costs, that are the foundation for software decay and death in all software investment. Architectural-level reasoning will lead to quantifiable improvements in the design and development of software resulting in higher quality, and sustainable software. Addressing software sustainability at the architectural level, allows the inhibiting or enabling of systems quality attributes, reasoning about and managing change as the system evolves, predicting system qualities, as well as measuring architecturally significant requirements. The ability to determine sustainability as a core software quality of a software system from an architectural perspective remains an open research challenge, and existing architectural principles need to be adapted and novel architectural paradigms devised. In addition, there is a pressing need for new tooling to fit today's emergent and dynamic environments, where software systems are explicitly designed for continuous evolvability and adaptability without incurring prohibitive architectural, technical debt (Northrop, 2018).

Acknowledgment

This work was partially supported by CNPq (313245/2021-5) and FAPESP (2015/24144-7).

References

Alharthi, A.D., Spichkova, M., Hamilton, M., 2018. Susoftpro: Sustainability profiling for software, in: 2018 IEEE 26th International Requirements Engineering Conference (RE), pp. 500–501.

Ali, N., Baker, S., O'Crowley, R., Herold, S., Buckley, J., 2018. Architecture consistency: State of the practice, challenges and requirements. Empirical Software Engineering 23, 224–258.

Amri, R., Ben Saoud, N.B., 2014. Towards a generic sustainable software model, in: 4th International Conference on Advances in Computing and Communications (ACC), pp. 231–234.

Andreessen, M., 2011. Why Software Is Eating The World. The Wall Street Journal URL: http://online.wsj.com/news/articles/SB10001424053111903480904576512250915629460.

Andrikopoulos, V., Boza, R.D., Perales, C., Lago, P., 2022. Sustainability in software architecture: A systematic mapping study, in: 2022 48th Euromicro Conference on Software Engineering and Advanced Applications (SEAA), pp. 426–433.

Andrikopoulos, V., Lago, P., 2021. Software Sustainability in the Age of Everything as a Service. Lecture Notes in Computer Science, pp. 35–47.

Atkins, T., Escudier, M., 2013. Durability, in: Dictionary of Mechanical Engineering. 1st ed.. Oxford University Press.

Autosar, 2023. Autosar. URL: http://www.autosar.org/.

Avgeriou, P., Stal, M., Hilliard, R., 2013. Architecture sustainability. IEEE Software 30, 40–44.

AWS, 2022. Sustainability Pillar. URL: https://docs.aws.amazon.com/wellarchitected/latest/framework/sustainability.html.

AXMEDIS, 2023. Axmedis. URL: http://www.axmedis.org/com/.

Azadi, U., et al., 2019. Architectural smells detected by tools: a catalogue proposal, in: IEEE/ACM International Conference on Technical Debt (TechDebt), pp. 88–97.

Bass, L., Clements, P., Kazman, R., 2021. Software Architecture in Practice. 4th ed., Addison-Wesley.

Becker, C., Chitchyan, R., Duboc, L., Easterbrook, S., Mahaux, M., Penzenstadler, B., Navas, G, R., Salinesi, C., Seyff, N., Venters, C.C., Calero, C., Ko¸cak, S.A., Betz, S., 2014. The Karlskrona manifesto for sustainability design. URL: http://sustainabilitydesign.org/.

Becker, C., et al., 2015. Sustainability Design and Software: The Karlskrona Manifesto, in: 37th IEEE International Conference on Software Engineering (ICSE), pp. 467–476.

Betz, S., Becker, C., Chitchyan, R., Duboc, L., Easterbrook, S., Penzenstadler, B., Seyff, N., Venters, C., 2015. Sustainability Debt: A Metaphor to Support Sustainability Design Decisions, in: 4th International Workshop on Requirements Engineering for Sustainable Systems @ 23rd IEEE International Requirements Engineering Conference (RE), pp. 55–63.

Betz, S., Duboc, L., Penzenstadler, B., Porras, J., Chitchyan, R., Seyff, N., Venters, C.C., Brooks, I., 2022. Susaf workbook 6.0. URL: https://doi.org/10.5281/zenodo.7342575.

Bijen, J., 2003. Durability of Engineering Structures Design, Repair and Maintenance. Woodhead Publishing.

Bischoff, Y., van der Wiel, R., van den Hooff, B., Lago, P., 2022. A taxonomy about information systems complexity and sustainability, in: Wohlgemuth, V., Naumann, S., Behrens, G., Arndt, H.K. (Eds.), Advances and New Trends in Environmental Informatics, Springer, Cham. pp. 17–33.

Boehm, B.W., 1988. A spiral model of software development and enhancement. Computer 21, 61–72.

Booch, G., 2006. The accidental architecture. IEEE Software 23, 9–11.

Brown, N.C.C., Sentance, S., Crick, T., Humphreys, S., 2014. Restart: The Resurgence of Computer Science in UK Schools. ACM Transactions on Computer Science Education 14, 1–22.

Cai, Y., 2010. Integrating sustainability into undergraduate computing education, in: 41st ACM Technical Symposium on Computer Science Education (SIGCSE'10), pp. 524–528.

Calero, C., Bertoa, M.F., Moraga, M.., 2013. A systematic literature review for software sustainability measures, in: 2nd International Workshop on Green and Sustainable Software (GREENS), pp. 46–53.

Calero, C., Moraga, M.Á., Piattini, M. (Eds.), 2021. Software Sustainability. Springer.

Capilla, R., Nakagawa, E.Y., Zdun, U., Carrillo, C., 2017. Toward architecture knowledge sustainability: Extending system longevity. IEEE Software 34, 108–111.

Carrillo, C., Capilla, R., 2018. Ripple effect to evaluate the impact of changes in architectural design decisions, in: P´erez, J., Mirandola, R., Chen, H. (Eds.), 12th European Conference on Software Architecture: Companion Proceedings, ECSA 2018, Madrid, Spain, September 24-28, 2018, pp. 41:1–41:8.

Carrillo, C., Capilla, R., Staron, M., 2020. Estimating the complexity of architectural design decision networks. IEEE Access 8, 168558–168575.

Cervantes, H., Kazman, R., 2016. Designing Software Architectures: A Practical Approach. Addison-Wesley.

Cohen, J., Katz, D.S., Barker, M., Chue Hong, N., Haines, R., Jay, C., 2021. The four pillars of research software engineering. IEEE Software 38, 97–105.

Condori-Fernández, N., Bagnato, A., Kern, E., 2018. A focus group for operationalizing software sustainability with the MEASURE platform, in: Condori-Fernández, N., Bagnato, A., Kern, E. (Eds.), 4th International Workshop on Measurement and Metrics for Green and Sustainable Software Systems co-located with Empirical Software Engineering International Week (ESEIW 2018), p. 7.

Condori-Fernández, N., Lago, P., 2018. Characterizing the contribution of quality requirements to software sustainability. J. Syst. Softw. 137, 289–305.

Creswell, J.W., Guetterman, T.C., 2020. Educational research: planning, conducting, and evaluating quantitative and qualitative research. 6th ed., Pearson Education Limited.

Crick, T., 2021. COVID-19 and Digital Education: A Catalyst for Change? ITNOW 63, 16–17.

Crick, T., 2022. An Introduction to Computer Science in the New Curriculum for Wales, in: 53rd ACM Technical Symposium on Computer Science Education (SIGCSE), p. 1142.

Crick, T., Davenport, J.H., Hanna, P., Irons, A., Prickett, T., 2020a. Computer Science Degree Accreditation in the UK: A Post-Shadbolt Review Update, in: 4th Conference on Computing Education Practice (CEP), pp. 1–4.

Crick, T., Hall, B.A., Ishtiaq, S., 2017. Reproducibility in Research: Systems, Infrastructure and Culture. Journal of Open Research Software 5, 1–9.

Crick, T., Knight, C., Watermeyer, R., 2022a. Measuring the impact of COVID-19 on the health and wellbeing of computer science practitioners, in: 53rd ACM Technical Symposium on Computer Science Education (SIGCSE), p. 1116.

Crick, T., Knight, C., Watermeyer, R., Goodall, J., 2020b. The Impact of COVID-19 and "Emergency Remote Teaching" on the UK Computer Science Education Community, in: UK and Ireland Computing Education Research Conference (UKICER), pp. 31–37.

Crick, T., Prickett, T., Bradnum, J., 2022b. A Preliminary Study of Peer Assessment Feedback within Team Software Development Projects, in: 53rd ACM Technical Symposium on Computer Science Education (SIGCSE'22).

Crick, T., Prickett, T., Bradnum, J., 2022c. Exploring Learner Resilience and Performance of First-Year Computer Science Undergraduate Students during the COVID-19 Pandemic, in: 27th Annual Conference on Innovation and Technology in Computer Science Education (ITiCSE), pp. 519–525.

Crick, T., Prickett, T., Bradnum, J., Godfrey, A., 2022d. Gender parity in peer assessment of team software development projects, in: Computing Education Practice (CEP'22).

Crick, T., Prickett, T., Vasiliou, C., Chitare, N., Watson, I., 2023. Exploring Computing Students Post-Pandemic Learning Preferences with Workshops: A UK Institutional Case Study, in: 28th Annual Conference on Innovation and Technology in Computer Science Education (ITiCSE'23).

Crouch, S., Hong, N.C., Hettrick, S., Jackson, M., Pawlik, A., Sufi, S., Carr, L., De Roure, D., Goble, C., Parsons, M., 2013. The Software Sustainability Institute: Changing Research Software Attitudes and Practices. Computing in Science & Engineering 15, 74–80.

Cutts, Q., Kallia, M., Anderson, R., Crick, T., Devlin, M., Farghally, M., Mirolo, C., Runde, R., Sepp ä l ä , O., Urquiza-Fuentes, J., Vahrenhold, J., 2023. Considering Computing Education in Undergraduate Computer Science Programmes, in: 28th Annual Conference on Innovation and Technology in Computer Science Education (ITiCSE'23).

Davenport, J.H., Crick, T., Hourizi, R., 2020. The Institute of Coding: A University-Industry Collaboration to Address the UK's Digital Skills Crisis, in: IEEE Global Engineering Education Conference (EDUCON), pp. 1400–1408.

Davenport, J.H., Hayes, A., Hourizi, R., Crick, T., 2016. Innovative Pedagogical Practices in the Craft of Computing, in: 4th International Conference on Learning and Teaching in Computing and Engineering (LaTiCE), pp. 115–119.

Duboc, L., Betz, S., Penzenstadler, B., Akinli Kocak, S., Chitchyan, R., Leifler, O., Porras, J., Seyff, N., Venters, C.C., 2019. Do we really know what we are building? raising awareness of potential sustainability effects of software systems in requirements engineering, in: 2019 IEEE 27th International Requirements Engineering Conference (RE), pp. 6–16.

Durdik, Z., Klatt, B., Koziolek, H., Krogmann, K., Stammel, J., Weiss, R., 2012. Sustainability guidelines for long-living software systems, in: 28th IEEE International Conference on Software Maintenance (ICSM), pp. 517–526.

Dwivedi, Y., Hughes, L., Ismagilova, E., Aarts, G., Coombs, C., Crick, T., Duan, Y., Dwivedi, R., Edwards, J., Eirug, A., Galanos, V., Ilavarasan, P.V., Janssen, M., Jones, P., Kumar Kar, A., Kizgin, H., Kronemann, B., Lal, B., Lucini, B., Medaglia, R., Le Meunier-FitzHugh, K., Le Meunier-FitzHugh, L., Misra, S., Mogaji, E., Sharma, S., Bahadur Singh, J., Raghavan, V., Raman, R., Rana, N., Samothrakis, S., Spencer, J., Tamilmani, K., Tubadji, A., Walton, P., Williams, M., 2021. Artificial Intelligence (AI): Multidisciplinary Perspectives on Emerging Challenges, Opportunities, and Agenda for Research, Practice and Policy. International Journal of Information Management 53.

Dwivedi, Y.K., Kshetri, N., Hughes, L., Slade, E., Jeyaraj, A., Kar, A., Baabdullah, A.M., Koohang, A., Raghavan, V., Ahuja, M., Al-Bashrawi, M., Al-Busaidi, A.S., Balakrishnan, J., Barlette, Y., Basu, S., Bose, I., Brooks, L., Buhalis, D., Carter, L., Chowdhury, S., Crick, T., Cunningham, S.W., Davies, G.H., Davison, R.M., D, R., Dennehy, D., Duan, Y., Dubey, R., Dutot, V., Dwivedi, R., Edwards, J.S., Flavin, C., Gauld, R., Grover, V., Hu, M.C., Janssen, M., Jones, P., Junglas, I., Khorana, S., Kraus, S., Larsen, K.R., Latreille, P., Laumer, S., Malik, F.T., Mardani, A., Mariani, M., Mithas, S., Mogaji, E., Nord, J., O'Connor, S., Okumus, F., Pagani, M., Pandey, N., Pappas, I.O., Pries-Heje, J., Papagiannidis, S., Pathak, N., Raman, R., Rana, N.P., Rehm, S.V., Ribeiro-Navarrete, S., Richter, A., Rowe, F., Sarker, S., Stahl, B., Tiwari, M., van der Aalst, W., Venkatesh, V., Viglia, G., Wade, M., Walton, P., Wirtz, J., Wright, R., 2023. "So what if ChatGPT wrote it?" Multidisciplinary perspectives on opportunities, challenges and implications of generative conversational AI for research, practice and policy. International Journal of Information Management 71.

EPSRC, 2020. Research software engineer fellowships 2020. URL: https://www.ukri.org/opportunity/research-software-engineer-fellowships-2020/. 17 Feb 2023.

EPSRC, 2021. Software for research communities. URL: https://www.ukri.org/opportunity/software-for-research-communities/. 17 Feb 2023.

Eveleens, J., Verhoef, C., 2010. The rise and fall of the chaos report figures. IEEE Software 27, 30–36.

Farley, D., 2021. Modern Software Engineering: Doing What Works to Build Better Software Faster. Addison-Wesley Professional.

Fatima, I., Lago, P., 2023. A review of software architecture evaluation methods for sustainability assessment, in: 2023 IEEE 20th International Conference on Software Architecture Companion (ICSA-C)., IEEE.

Felizardo, K.R., Mendes, E., Kalinowski, M., Souza, E.F., Vijaykumar, N.L., 2016. Using forward snowballing to update systematic reviews in software engineering, in: 10th ACM/IEEE International Symposium on Empirical Software Engineering and Measurement.

Fisher, D.H., Bian, Z., Chen, S., 2016. Incorporating Sustainability into Computing Education. IEEE Intelligent Systems 31, 93–96.

Forsberg, K., Mooz, H., 1991. The relationship of system engineering to the project cycle. INCOSE International Symposium 1, 57–65.

Frezza, S.T., Tang, M.H., Rowland, S., 2019. Understanding the cost of change: Measuring sustainability of computing education, in: IEEE Frontiers in Education Conference (FIE), pp. 1–8.

Funke, M., Lago, P., Verdecchia, R., 2023. Variability features: Extending sustainability decision maps via an industrial case study, in: 2023 IEEE 20th International Conference on Software Architecture Companion (ICSA-C).

Future Generations Commissioner for Wales, 2015. Well-being of Future Generations (Wales) Act 2015. URL: https://www.futuregenerations.wales/about-us/future-generations-act/.

Garc´es, L., Martnez-Fernndez, S., Oliveira, L., Valle, P., Ayala, C., Franch, X., Nakagawa, E., 2021. Three decades of software reference architectures: A systematic mapping study. Journal of Systems and Software 179, 1–27.

Garc´ıa-Berná, J.A., Ouhbi, S., Fernández-Alemán, J.L., 2022. Assessing software sustainability ofconnected health applications, in: Rocha, A., Adeli, H., Dzemyda, G., Moreira, F. (Eds.), Information Systems and Technologies, Springer International Publishing. pp. 498–509.

Garc´ıa-Mireles, G.A., ngeles Moraga, M., Garca, F., Calero, C., Piattini, M., 2018. Interactions between environmental sustainability goals and software product quality: A mapping study. Information and Software Technology 95, 108–129.

Garc´ıa-Mireles, G.A., Villa-Mart´ınez, H.A., 2018. Practices for addressing environmental sustainability through requirements processes, in: International Conference on Software Process Improvement (CIMPS), pp. 61–70.

Garlan, D., 2000. Software architecture: A roadmap, in: Conference on The Future of Software Engineering (ICSE), pp. 91–101.

Garlan, D., Bachmann, F., Ivers, J., Stafford, J., Bass, L., Clements, P., Merson, P., 2010. Documenting Software Architectures: Views and Beyond. 2nd ed., Addison-Wesley Professional.

Gharbi, M., Koschel, A., Rausch, A., 2019. Software Architecture Fundamentals. Dpunkt.Verlag.

Giray, G., Catal, C., 2021. Design of a Data Management Reference Architecture for Sustainable Agriculture. Sustainability 13, 1–17.

Glavic, P., Lukman, R., 2007. Review of sustainability terms and their definitions. Journal of Cleaner Production 15, 1875–1885.

Goble, C., 2014. Better software, better research. IEEE Internet Computing 18, 4–8.

Google, 2022. Design for environmental sustainability. URL: https://cloud.google.com/architecture/framework/system-design/sustainability.

Gradinik, M., Berani, T., Karakati, S., 2020. Impact of historical software metric changes in predicting future maintainability trends in open-source software development. Applied Sciences 10.

Groher, I., Weinreich, R., 2017. An interview study on sustainability concerns in software development projects, in: 43rd Euromicro Conference on Software Engineering and Advanced Applications (SEAA), pp. 350–358.

Grua, E.M., De Sanctis, M., Lago, P., 2020. A Reference Architecture for Personalized and Self-adaptive e-Health Apps, Springer. Communications in Computer and Information Science, pp. 195–209.

Guamán, D., P´erez, J., 2021. Supporting sustainability and technical debt-driven design decisions in software architectures, in: Insfrán, E., Abrahão, S., Fernández, M., Barry, C., Lang, M., Linger, H., Schneider, C. (Eds.), Information Systems Development: Crossing Boundaries between Development and Operations (DevOps) in Information Systems (ISD2021 Proceedings), Valencia, Spain, September 8-10, 2021, Universitat Polit`ecnica de Val`encia / Association for Information Systems.

Gupta, S., Lago, P., Donker, R., 2021. A framework of software architecture principles for sustainability-driven design and measurement, in: 2021 IEEE 18th International Conference on Software Architecture Companion (ICSA-C), Institute of Electrical and Electronics Engineers Inc., United States. pp. 31–37. doi:10.1109/ICSA-C52384.2021.00012. publisher Copyright: © 2021 IEEE. Copyright: Copyright 2021 Elsevier B.V., All rights reserved.; 18th IEEE International Conference on Software Architecture Companion, ICSA-C 2021 ; Conference date: 22-03-2021 Through 26-03-2021.

Hahner, U.R., Balduzzi, G., Doak, P.W., Maier, T.A., Solca, R., Schulthess, T.C., 2019. Dca++ project: Sustainable and scalable development of a high-performance research code. Journal of Physics: Conference Series 1290, 012017.

Hardman, J., Watermeyer, R., Shankar, K., Suri, V., Crick, T., Knight, C., McGaughey, F., Chung, R., 2022. "Does anyone even notice us?" COVID-19's impact on academics' well-being in a developing country. South African Journal of Higher Education 36, 1–19.

Heldal, R., Nguyen, N.T., Moreira, A., Lago, P., Duboc, L., Betz, S., Coroama, V., Penzenstadler, B., Porras, J., Oyedeji, S., Capilla, R., Brooks, J., Venters, C., 2023. Sustainability Competencies and Skills in Software Engineering: An Industry Perspective. WorkingPaper.

Hintemann, R., Hinterholzer, S., 2019. Energy consumption of data centers worldwide - how will the internet become green?, in: 6th International Conference on ICT for Sustainability (ICT4S), pp. 1–8.

Ioannidis, Y., 2023. To the Members of ACM. Communications of the ACM 66, 5.

Irons, A., Crick, T., 2022. in: Higher Education in a Post-COVID World: New Approaches and Technologies for Teaching and Learning. Emerald Publishing. chapter Cybersecurity in the Digital Classroom: Implications for Emerging Policy, Pedagogy and Practice, pp. 231–244.

Irons, A., Crick, T., Davenport, J.H., Hanna, P., Prickett, T., 2021. Increasing the Value of Professional Body Computer Science Degree Accreditation, in: 52nd ACM Technical Symposium on Computer Science Education (SIGCSE'21).

ISO/IEC/IEEE, 2017. ISO/IEC/IEEE 24765:2017(E) International Standard - Systems and software engineering - Vocabulary. URL: https://www.iso.org/standard/71952.html.

Jacobson, I., 2018. 50 years of SE, so now what? 40th International Conference on Software Engineering (ICSE).

Jacobson, I., Booch, G., Rumbaugh, J., 1999. The Unified Software Development Process. Addison-Wesley.

Jansen, A., Bosch, J., Avgeriou, P., 2008. Documenting after the fact: Recovering architectural design decisions. Journal of Systems and Software 81, 536–557.

Johnson, J., 2021. CHAOS Report: Beyond Infinity. Technical Report. The Standish Group.

Kakamoukas, G., Sarigiannidis, P., Maropoulos, A., Lagkas, T., Zaralis, K., Karaiskou, C., 2021. Towards Climate Smart Farming - A Reference Architecture for Integrated Farming Systems. Telecom 2, 52–74.

Karanikiotis, T., Papamichail, M.D., Symeonidis, A.L., 2021. Analyzing static analysis metric trends towards early identification of non-maintainable software components. Sustainability 13.

Kitchenham, B., Pearl Brereton, O., Budgen, D., Turner, M., Bailey, J., Linkman, S., 2009. Systematic literature reviews in software engineering a systematic literature review. Information and Software Technology 51, 7–15.

Koedijk, L., Oprescu, A., 2022. Finding significant differences in the energy consumption when comparing programming languages and programs, in: 8th International Conference on ICT for Sustainability (ICT4S), pp. 1–12.

Koziolek, H., 2011a. Sustainability evaluation of software architectures: A systematic review, in: 7th International Conference on the Quality of Software Architectures (QoSA) and 2nd International Symposium on Architecting Critical Systems (ISARCS), pp. 3–12.

Koziolek, H., 2011b. Sustainability evaluation of software architectures: A systematic review, in: Joint ACM SIGSOFT Conference on Quality of Software Architectures & ACM SIGSOFT Symposium Architecting Critical Systems (QoSA/ISARCS), pp. 3–12.

Koziolek, H., Domis, D., Goldschmidt, T., Vorst, P., 2013. Measuring architecture sustainability. IEEE Software 30, 54–62.

Koziolek, H., Domis, D., Goldschmidt, T., Vorst, P., Weiss, R.J., 2012. MORPHOSIS: A lightweight method facilitating sustainable software architectures, in: Joint Working IEEE/IFIP Conference on Software Architecture and European Conference on Software Architecture (WICSA/ECSA), pp. 253–257.

KPIT Technologies Ltd, 2022. Electric and Conventional Powertrain. URL: https://www.kpit.com/solutions/electric-powertrain-offerings/.

Lachney, M., Ryoo, J., Santo, R., 2021. Introduction to the Special Section on Justice-Centered Computing Education, Part 1. ACM Transactions on Computing Education 21, 1–15.

Lago, P., Greefhorst, D., Woods, E., 2022. Architecting for sustainability, in: Wohlgemuth, V., Naumann, S., Arndt, H.K., Behrens, G., Hb, M. (Eds.), EnviroInfo 2022, p. 199.

Lago, P., Kazman, R., Meyer, N., Morisio, M., Müller, H.A., Paulisch, F., Scanniello, G., Penzenstadler, B., Zimmermann, O., 2013. Exploring initial challenges for green software engineering: summary of the first GREENS workshop, at ICSE 2012. ACM SIGSOFT Software Engineering Notes 38, 31–33.

Lago, P., Verdecchia, R., Condori Fernandez, N., Rahmadian, E., Sturm, J., van Nijnanten, T., Bosma, R., Debuysscher, C., Ricardo, P., 2020. Designing for sustainability: Lessons learned from four industrial projects. URL: http://cyprusconferences.org/enviroinfo2020. international Conference on Informatics for Environmental Protection, EnviroInfo ; Conference date: 23-09-2020 Through 25-09-2020.

Lautenschutz, D., España, S., Hankel, A., Overbeek, S., Lago, P., 2018. A comparative analysis of green ICT maturity models, in: 5th International Conference on Information and Communication Technology for Sustainability (ICT4S), pp. 153–167.

Le, D.M., Carrillo, C., Capilla, R., Medvidovic, N., 2016. Relating architectural decay and sustainability of software systems, in: 13th Working IEEE/IFIP Conference on Software Architecture (WICSA), pp. 178–181.

Lilienthal, C., 2019. Sustainable Software Architecture: Analyze and Reduce Technical Debt. Rocky Nook.

Mancebo, J., Calero, C., Garc´ıa, F., 2021a. Does maintainability relate to the energy consumption of software? A case study. Software Quality Journal 29, 101–127.

Mancebo, J., Calero, C., Garc´ıa, F., Moraga, M.Á., de Guzmán, I.G.R., 2021b. FEETINGS: framework for energy efficiency testing to improve environmental goal of the software. Sustainable Computing: Informatics and Systems 30, 100558.

Martin, K., 2022. RSE-at-SUN: Research Software Engineering (RSE) Group at Stellenbosch University, South Africa. URL: https://rse-at-sun.github.io/RSE-at-SUN/.

Martin, R.C., 2017. Clean Architecture: A Craftsman's Guide to Software Structure and Design. 1st ed., Prentice Hall Press, USA.

McGaughey, F., Watermeyer, R., Shankar, K., Suri, V., Knight, C., Crick, T., Hardman, J., Phelan, D., Chung, R., 2022. 'This can't be the new norm': academics' perspectives on the COVID-19 crisis for the Australian University Sector. Higher Education Research & Development 41.

McGuire, S., Shultz, E., Ayoola, B., Ralph, P., 2023. Sustainability is stratified: Toward a better theory of sustainable software engineering. arXiv:2301.11129.

Microsoft, 2022. Sustainable workloads. URL: https://learn.microsoft.com/en-us/azure/architecture/framework/sustainability/sustainabi

Mirakhorli, M., 2015. Software architecture reconstruction: Why? what? how?, in: IEEE 22nd International Conference on Software Analysis, Evolution, and Reengineering (SANER), pp. 595--595.

Moises, A.C., Malucelli, A., Reinehr, S., 2018. Practices of energy consumption for sustainable software engineering, in: 9th International Green and Sustainable Computing Conference (IGSC), pp. 1--6.

Moller, F., Crick, T., 2018. A University-Based Model for Supporting Computer Science Curriculum Reform. Journal of Computers in Education 5, 415--434.

Mourao, B.C., Karita, L., do Carmo Machado, I., 2018. Green and sustainable software engineering - a systematic mapping study, in: XVII Brazilian Symposium on Software Quality, Association for Computing Machinery. pp. 121--130. URL: https://doi.org/10.1145/3275245.3275258, doi:10.1145/3275245.3275258.

Murphy, E., Crick, T., Davenport, J.H., 2017. An Analysis of Introductory Programming Courses at UK Universities. The Art, Science, and Engineering of Programming 1(2), 1--23.

Nakagawa, E., Antonino, P., 2023. Reference Architectures for Critical Domains: Industrial Uses and Impacts. 1 ed., Springer.

Nakagawa, E.Y., Antonino, P.O., Becker, M., 2011. Reference architecture and product line architecture: A subtle but critical difference, in: 5th European Conference on Software Architecture (ECSA), pp. 207--211.

Nakagawa, E.Y., Antonino, P.O., Schnicke, F., Capilla, R., Kuhn, T., Liggesmeyer, P., 2021. Industry 4.0 reference architectures: State of the art and future trends. Computers & Industrial Engineering 156, 1--13.

NATO, 1968. IEEE Computer Society Press.

Naumann, S., Dick, M., Kern, E., Johann, T., 2011. The greensoft model: A reference model for green and sustainable software and its engineering. Sustainable Computing: Informatics and Systems 1, 294--304.

Nazir, S., Fatima, N., Chuprat, S., Sarkan, H., F, N., Sjarif, N.N., 2020. Sustainable software engineering: A perspective of individual sustainability. International Journal on Advanced Science, Engineering and Information Technology 10, 676--683.

Northrop, L., 2018. Modern trends through an architecture lens, in: 40th International Conference on Software Engineering (ICSE).

NSF, 2022. Design for sustainability in computing. URL: https://beta.nsf.gov/funding/opportunities/design-sustainability-computing. 17 Feb 2023.

OED, 2023. Oxford English Dictionary. Oxford University Press.

Ojameruaye, B., Bahsoon, R., Duboc, L., 2016. Sustainability Debt: A Portfolio-Based Approach for Evaluating Sustainability Requirements in Architectures, in: 38th International Conference on Software Engineering Companion (ICSE), pp. 543--552.

Ousterhout, J., 2021. A Philosophy of Software Design. Yaknyam Press.

Oyedeji, S., Seffah, A., Penzenstadler, B., 2018. Classifying the measures of software sustainability according to the current perceptions, in: 4th International Workshop on Measurement and Metrics for Green and Sustainable Software Systems (MeGSuS) @ Empirical Software Engineering International Week (ESEIW 2018), p. 19.

Parnas, D., 1972. On the criteria to be used in decomposing systems into modules. Communications of the ACM 15, 1053--1058.

Paulauskaite-Taraseviciene, A., Lagzdinyte-Budnike, I., Gaiziuniene, L., Sukacke, V., Daniuseviciute-Brazaite, L., 2022. Assessing Education for Sustainable

Development in Engineering Study Programs: A Case of AI Ecosystem Creation. Sustainability 14, 1--22. URL: https://ideas.repec.org/a/gam/jsusta/v14y2022i3p1702-d740381.html.

Penzenstadler, B., 2013. Towards a definition of sustainability in and for software engineering, in: 28th Annual ACM Symposium on Applied Computing (SAC), pp. 1183--1185.

Penzenstadler, B., Raturi, A., Richardson, D.J., Calero, C., Femmer, H., Franch, X., 2014. Systematic mapping study on software engineering for sustainability (SE4S), in: 18th International Conference on Evaluation and Assessment in Software Engineering (EASE), pp. 14:1--14:14.

Peters, A.K., Capilla, R., Coroama, V.C., Heldal, R., Lago, P., Leifler, O., Moreira, A., Fernandes, J.P., Penzenstadler, B., Porras, J., Venters, C.C., 2023. Sustainability in computing education: A systematic literature review arXiv:2305.10369.

Pham, Y.D., Bouraffa, A., Hillen, M., Maalej, W., 2021. The role of linguistic relativity on the identification of sustainability requirements: An empirical study, in: 2021 IEEE 29th International Requirements Engineering Conference (RE), pp. 117--127.

Pham, Y.D., Bouraffa, A., Maalej, W., 2020. Shapere: Towards a multi-dimensional representation for requirements of sustainable software, in: IEEE 28th International Requirements Engineering Conference (RE), pp. 358--363.

Prickett, T., Harvey, M., Walters, J., Yang, L., Crick, T., 2020. Resilience and Effective Learning in First-Year Undergraduate Computer Science, in: 25th Annual Conference on Innovation and Technology in Computer Science Education (ITiCSE), pp. 19--25.

Prickett, T., Yang, L., Irons, A., Miller, K., Brooke, P., Crick, T., Hayes, A., Davenport, J.H., English, R., Maguire, J., Bechkoum, K., Jones, A., 2023. Challenges and Opportunities of Teaching Cybersecurity in UK University Computing Programmes, in: Sikos, L.F., Haskell-Dowland, P. (Eds.), Cybersecurity Teaching in Higher Education, pp. 1--35.

Royce, W.W., 1987. Managing the development of large software systems: Concepts and techniques, in: I9th international conference on Software Engineering (ICSE), pp. 328--388.

Santana, E., Chaves, A., Gerosa, M., Kon, F., Milojicic, D., 2018. Software Platforms for Smart Cities: Concepts, Requirements, Challenges, and a Unified Reference Architecture. ACM Computing Surveys 50, 1--37.

Schwaber, K., 1997. Scrum development process, in: Sutherland, J., Casanave, C., Miller, J., Patel, P., Hollowell, G. (Eds.), Business Object Design and Implementation, pp. 117--134.

Sdoukopoulos, A., et al., 2019. Measuring progress towards transport sustainability through indicators: Analysis and metrics of the main indicator initiatives. Transportation Research Part D: Transport and Environment 67, 316--333.

Seacord, R., Elm, J., Goethert, W., Lewis, G., Plakosh, D., Robert, J., Wrage, L., Lindvall, M., 2003. Measuring software sustainability, in: International Conference on Software Maintenance (ICSM), pp. 450--459.

SEI, 1984. Software Engineering Institute. URL: https://www.sei.cmu.edu. 17 Feb 2023.

Sentance, S., Kirby, D., Quille, K., Cole, E., Crick, T., Looker, N., 2022. Computing in School in the UK & Ireland: A Comparative Study, in: UK and Ireland Computing Education Research Conference (UKICER'22).

Seyff, N., Betz, S., Groher, I., Stade, M., Chitchyan, R., Duboc, L., Penzenstadler, B., Venters, C., Becker, C., 2018. Crowd-focused semi-automated requirements engineering for evolution towards sustainability, in: 2018 IEEE 26th International Requirements Engineering Conference (RE), pp. 370--375.

Seyff, N., et al., 2022. Transforming our world through software: Mapping the sustainability awareness framework to the un sustainable development goals, in: 17th International Conference on Evaluation of Novel Approaches to Software Engineering (ENASE), pp. 417--425.

Sharma, T., Singh, P., Spinellis, D., 2020. An empirical investigation on the relationship between design and architecture smells. Empirical Software Engineering 25, 4020--4068.

Sharma, T., Spinellis, D., 2018. A survey on software smells. Journal of Systems and Software 138, 158--173.

Shaw, M., 2016. Progress toward an engineering discipline of software, in: 38th International Conference on Software Engineering Companion (ICSE-C), pp. 3--4.

Siegel, A., Zarb, M., Alshaigy, B., Blanchard, J., Crick, T., Glassey, R., Holt, J.R., Latulipe, C., Riedesel, C., Senapathi, M., Simon, Williams, D., 2021. Teaching through a Global Pandemic: Educational Landscapes Before, During and After COVID-19, in: 2021 Working Group Reports on Innovation and Technology in Computer Science Education (ITiCSE-WGR), pp. 1--25.

Simon, Mason, R., Crick, T., Davenport, J.H., Murphy, E., 2018. Language Choice in Introductory Programming Courses

at Australasian and UK Universities, in: 49th ACM Technical Symposium on Computer Science Education (SIGCSE'18), pp. 852--857.

Smith, A.M., Katz, D.S., Niemeyer, K.E., FORCE11 Software Citation Working Group, 2016. Software Citation Principles. PeerJ Computer Science 2, 1--31.

SRSE, 2013. Society of research software engineering. URL: https://society-rse.org. 17 Feb 2023.

SSI, 2010. Software sustainability institute. URL: https://www.software.ac.uk. 17 Feb 2023.

Standish Group International, I., 2020. CHAOS Report: Beyond Infinity. Technical Report.

Sufi, S., Jay, C., 2018. Raising the status of software in research: A survey-based evaluation of the Software Sustainability Institute Fellowship Programme [version 1; peer review: 3 approved with reservations]. F1000Research 7.

Sverdrup, H., Svensson, M.G.E., 2005. Defining the concept of sustainability - A matter of systems thinking and applied systems analysis, in: Olsson, M.O., Sjöstedt, G. (Eds.), Systems Approaches and Their Application: Examples from Sweden. Springer, pp. 143--164.

Swacha, J., Maskelinas, R., Damaeviius, R., Kulikajevas, A., Blaauskas, T., Muszyska, K., Miluniec, A., Kowalska, M., 2021. Introducing Sustainable Development Topics into Computer Science Education: Design and Evaluation of the Eco JSity Game. Sustainability 13, 1--17.

Tamburri, D.A., Kazman, R., 2018. General methods for software architecture recovery: a potential approach and its evaluation. Empirical Software Engineering 23, 1457--1489.

Tanveer, B., 2021. Sustainable software engineering -- have we neglected the software engineer's perspective?, in: 2021 36th IEEE/ACM International Conference on Automated Software Engineering Workshops (ASEW), pp. 267--270.

Taylor, R.N., Medvidovic, N., Dashofy, E., 2009. Software Architecture: Foundations, Theory, and Practice. Wiley.

Thomas, C., 2016. Complex Systems, Sustainability and Innovation. IntechOpen.

Tryfonas, T., Crick, T., 2015. Smart Cities, Citizenship Skills and the Digital Agenda: The Grand Challenges of Preparing the Citizens of the Future. Technical Report. UK Government Office for Science and Department for Business, Innovation & Skills. URL: https://www.gov.uk/government/publications/future-of-cities-smart-cities-citizenship-ski

US Department of Transportation, 2022. Architecture Reference for Cooperative and Intelligent Transportation (ARC-IT). URL: https://local.iteris.com/arc-it/html/architecture/architecture.html.

Venters, C., 2021. Is sustainable software that which is explicitly designed for continuous maintainability and evolvability without incurring prohibitive technical debt and a negative impact on the dimensions of sustainability? discuss. URL: https://twitter.com/ccv1968/status/1400088407963996161. 17 Feb 2023.

Venters, C., et al., 2018. Software sustainability: Research and practice from a software architecture viewpoint. Journal of Systems and Software 138, 174--188.

Venters, C., et al., 2021. Software Sustainability: Beyond the Tower of Babel, in: IEEE/ACM International Workshop on Body of Knowledge for Software Sustainability (BoKSS), pp. 3--4.

Venters, C.C., Lau, L., Griffiths, M.K., Holmes, V., Ward, R.R., Jay, C., Dibsdale, C.E., Xu, J., 2014. The blind men and the elephant: Towards an empirical evaluation framework for software sustainability. Journal of Open Research Software 2, 1--6.

Voas, J.M., Kuhn, R., 2017. What happened to software metrics? Computer 50, 88--98.

Volpato, T., Allian, A., Nakagawa, E.Y., 2019. Has social sustainability been addressed in software architectures?, in: 13th European Conference on Software Architecture (ECSA-C), pp. 245--249.

Volpato, T., Oliveira, B.R.N., Garcés, L., Capilla, R., Nakagawa, E.Y., 2017. Two perspectives on reference architecture sustainability, in: 11th European Conference on Software Architecture Workshops (ECSAW 2017), pp. 188--194.

Wang, Z., Yin, D., 2019. Design and Implementation of Vehicle Control System for Pure Electric Vehicle Based on AUTOSAR Standard, in: 22nd International Conference on Electrical Machines and Systems (ICEMS), pp. 1--5.

Ward, R., Crick, T., Davenport, J.H., Hanna, P., Hayes, A., Irons, A., Miller, K., Moller, F., Prickett, T., Walters, J., 2023. Using skills profiling to enable badges and micro-credentials to be incorporated into higher education courses. Journal of Interactive Media in Education 2023(1), 1317--1336.

Ward, R., Phillips, O., Bowers, D., Crick, T., Davenport, J.H., Hanna, P., Hayes, A., Irons, A., Prickett, T., 2021. Towards a 21st Century Personalised Learning Skills Taxonomy, in: IEEE Global Engineering Education Conference (EDUCON), pp. 344--354.

Watermeyer, R., Crick, T., Knight, C., 2022. Digital disruption in the time of COVID-19: Learning technologists' accounts of institutional barriers to online learning, teaching and assessment in UK universities. International Journal for Academic Development 27, 148--162.

Watermeyer, R., Crick, T., Knight, C., Goodall, J., 2021a. COVID-19 and digital disruption in UK universities: afflictions and affordances of emergency online migration. Higher Education 81, 623--641.

Watermeyer, R., Shankar, K., Crick, T., Knight, C., McGaughey, F., Hardman, J., Suri, V., Chung, R., Phelan, D., 2021b. 'Pandemia': A reckoning of UK universities' corporate response to COVID-19 and its academic fallout. British Journal of Sociology of Education 42, 651--666.

Wilson, G., 2014. Software Carpentry: lessons learned [version 2; peer review: 3 approved]. F1000Research 3.

Wohlin, C., 2014. Guidelines for snowballing in systematic literature studies and a replication in software engineering, in: 18th International Conference on Evaluation and Assessment in Software Engineering, Association for Computing Machinery.

Wohlin, C., Kalinowski, M., Felizardo, K.R., Mendes, E., 2022. Successful combination of database search and snowballing for identification of primary studies in systematic literature studies. Information and Software Technology 147.

von Zitzewitz, A., 2022. Using software metrics to ensure maintainability, in: Ciceri, C., Farley, D., Ford, N., Harmel-Law, A., Keeling, M., Lilienthal, C., Rosa, J., von Zitzewitz, A., Weiss, R., Wood, E. (Eds.), Software Architecture Metrics: Case Studies to Improve the Quality of Your Architecture. O'Reilly Media, pp. 143--171.